

어선 규모(대형) 예측 — 작성 보고서

0 기본 정보

이름 김정희 조 2 제출일 2026-07-21 GitHub REPO <https://github.com/2-NCS/2-DBMS>

과정명	(디지털 컨버전스) 공공데이터 융합 풀스택 개발자 양성과정E
능력단위	공공데이터 융합 DBMS 구축 / 7수준
평가방법	문제해결시나리오 (100점)
평가일자	2026-07-21 (09:00 ~ 18:00)

제출방법

PPT 제출 (실행화면 캡처 포함) · 소스 폴더(01 / 02 / 03) GITHUB REPO 링크 제출

1 공공데이터 검색·확보

1-1 어떤 데이터를 골랐는가

데이터셋명 어선검사현황

공공데이터포털 URL <https://www.data.go.kr/data/15049852/fileData.do>

검색에 쓴 키워드 검사현황 확장자 필터 CSV

1-2 왜 이 데이터를 골랐는가 (선정 이유)

선택한 선박이 소형인지 대형인지 예측하기 위해, 한국해양교통안전공단_어선검사현황 데이터를 선정했습니다.

1-3 데이터 선정 조건 3가지를 만족하는지 확인

행 단위 데이터인가(1행 = 어선 1척) 예

타겟이 두 가지 값(이진분류)이 되는가 예

통계·집계표가 아닌가 예

2 구조·유형·척도 파악

2-1 구조 — 실행 결과를 보고 채운다

```
df = pd.read_csv('data/어선검사항황.csv', encoding='cp949')
print('데이터 크기(행, 열):', df.shape)
```

행 63647 개 열 11 개

2-2 주요 열의 유형·척도

열 이름	유형(문자/숫자/날짜)	척도(명목/순서/등간/비율)	입력으로 쓸까
선질	문자	명목	사용
용도	문자	명목	사용(상위 20 + 기타)
선적항	문자	명목	시도만 추출해 사용
총톤수	숫자	비율	제외(타겟 재료)
길이	숫자	비율	제외(누수)

2-3 분석 가능하다고 판단한 근거

총 행수는 63647 건으로 충분하다 판단되었고, 입력으로 사용될 열로는 선질, 용도, 선적항을 선택했습니다. 타겟으로는 총톤수를 기준으로 0, 1로 소형 대형을 구분했습니다

3 출처·제공기관·갱신주기·이용허락범위

찾는 곳

공공데이터포털 데이터 상세페이지의 **파일데이터 정보** 표에 제공기관·갱신주기·이용허락범위가 모두 적혀 있다.

3-1 상세페이지에서 그대로 옮겨 적는다

출처	공공데이터포털(data.go.kr) — 한국해양교통안전공단_어선검사현황_20260617 [15049852]
제공기관	한국해양교통공단
갱신주기	연간
이용허락범위	제한 없음
전체 건수	63647
등록일 / 수정일	2026-06-23/2026-06-24

4 품질 진단

배점이 가장 큰 항목

네 가지를 모두 수치로 제시해야 한다. "없었다" 도 확인해 봤다는 근거(코드·출력)가 있어야 인정된다.

4-1 결측치 — 몇 개인지 세어서 적는다

```
print(df.isnull().sum())
```

결측이 있는 열과 개수(비율) 결측 없음 — isnull().sum() 모두 0임을 확인

처리 방향 그대로 두기

4-2 이상치 — 수치형 열의 분포를 보고 판단한다

```
print(df[['총톤수', '길이', '너비', '깊이']].describe()) # 최소·최대·사분위수 확인
```

이상해 보이는 값이 있는 열과 근거 총톤수 중앙값 4.5t 대비 수백 t 급이 있으나 실제 대형 선박이라 오류가 아니다

처리 방향(제거 / 유지 / 상한 자르기)과 이유 유지 — 어차피 누수라 입력에서 제외하는 열이다

4-3 표기 불일치 — 같은 것이 두 이름으로 적혀 있지 않은가

```
print(df[' 시도 '].value_counts()) # 값을 전부 찍어 눈으로 확인
```

불일치 사례 '충남'(6,102)과 '충청남도'(1,012)가 따로 집계됨. 부산시/부산광역시 등 6쌍이 같은 문제

통일 후 범주 수 19 종 → 12 종

4-4 희소 범주 — 몇 건 안 되는 범주가 많지 않은가

```
for c in ['선질', '용도', '세부용도', '시도']:
    print(c, df[c].nunique(), '종')
```

범주가 많은 열과 종 수 선질 5종, 용도 59종, 세부용도 84종, 선적항 450종

묶는 기준(상위 N개 + 기타) 용도·세부용도는 상위 20개 + 기타, 선적항은 시도만 추출(450종 → 12종)

4-5 정제 방향 정리

isnull().sum()으로 결측치 없이 모두 0임을 확인했고, 이상치로는 총톤수 중앙값 4.5대비 수백 t급이 있으나 실제 대형 선박으로 분류해 유지하는 방향으로 진행합니다. 표기 불일치는 충남, 충청남도처럼 같은 지역이지만 따로 표기되는건 표기 통일로 종을 줄이고, 몇 건 안되는 희소 범주는 '기타'로 분류해 한곳에 모읍니다.

5 타겟 정의·독립변수 선별

5-1 타겟(종속변수) 정의

```
TARGET = ' 대형 '
LABELS = [' 소형 ', ' 대형 ' ]
df[TARGET] = ( df['총톤수'] > MED ).astype(int)
```

정의 기준을 말로 쓰면 _____

5-2 클래스 분포 — 균형인가 불균형인가

0(소형) 31886 건 1(대형) 31761 건 양성 비율 49.90 %

이 분포가 뒤(분할·학습·지표)에 미치는 영향 균형이라 정확도를 봐도 된다. 기준선이 0.5 다. class_weight도 필요 없다

5-3 데이터 누수가 되는 컬럼을 제외한 근거

누수 주의

타겟을 구성하는 컬럼과 사건 종료 후 확정되는 사후 결과값은 **반드시** 입력에서 뺀다. ROC-AUC 가 0.99 처럼 비정상적으로 높으면 누수를 의심한다.

뺀 컬럼 총톤수, 길이, 넓이, 깊이

뺀 이유 총톤수는 타겟을 만든 열이고, 치수는 총톤수를 계산하는 값이라 사실상 정답이다. 실제로 치수를 넣으면 ROC-AUC 가 0.9987 로 치솟고 길이 하나가 중요도 0.546 을 차지한다

5-4 남긴 독립변수와 선정 근거

최종 입력 3 개 : material, usage, port

material, usage, port은 모두 선박이 등록되는 시점에 이미 결정되는 값으로, 실제 운항 데이터나 총톤수 측정 이후에 발생하는 정보가 아닙니다. 즉 예측 시점에 이미 알 수 있는 입력값이므로 데이터 누수 우려가 없습니다.

6 정제·가공하여 분석 가능한 형태로 구축

6-1 표기 통일

```
PORT = {'충남': '충청남도', '부산시': '부산광역시', '제주시': '제주도', ...}
df[' 시도 '] = df[' 시도 '].replace(PORT)
```

결과 19 종 → 12 종

6-2 희소 범주 묶기

```
top = df[' 용도 '].value_counts().head(20).index
df[' 용도 G '] = np.where(df[' 용도 '].isin(top), df[' 용도 '], '기타')
```

6-3 인코딩 — 글자를 숫자로

중요

코드 열 이름은 **영문**으로 만든다. 이 이름이 그대로 API의 JSON 키가 되는데 한글 키는 터미널·도구에
서 인코딩이 깨져 오류가 난다. 보이는 이름은 `label` 이 담당한다.

인코딩 순서는 `feature_spec.json` 의 `options` 배열과 반드시 일치해야 한다. 화면은 선택지의 인
덱스를 보내기 때문이다.

```
OPTIONS[c] = sorted(df[c].astype(str).unique())
df[CODE[c]] = df[c].astype(str).map({v: i for i, v in enumerate(OPTIONS[c])})
```

정렬 함수 · 인덱스 부여 함수를 채운 이유 options 순서와 코드가 어긋나면 화면에서 고른 것과 다른 값으로
예측된다

6-4 8:2 분할 — 클래스 분포를 고려한다

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

학습 50917 행 / 검증 12730 행

stratify 를 쓴 이유 균형 데이터라 없어도 큰 차이는 없지만, 검증의 클래스 비율을 학습과 똑같이 맞춰 두는
것이 안전하다

6-5 결측치 채우기는 언제 하는가

누수 주의

중앙값·평균으로 채울 때 **전체 데이터의 중앙값**을 쓰면 검증 데이터의 정보가 학습에 새어 든다. 반드시 **분할한 뒤 학습 데이터의 값**으로 채운다.

채우기를 분할 후 에 한 이유 결측이 없어 채우기 자체가 필요 없다

6-6 학습

```
model = RandomForestClassifier(n_estimators=100, max_depth=8,
                               random_state=42 (필요 없어 비워 둬))
model.fit(X_tr, y_tr)
```

class_weight 를 쓴 / 쓰지 않은 이유 타겟이 49.9 : 50.1 로 균형이라 필요 없다. 주면 오히려 아무 이득 없이 결정 기준만 흐른다

6-7 성능지표 4종과 지표 선택의 근거

```
acc = accuracy_score(y_te, pred)
auc = roc_auc_score(y_te, proba)
print(classification_report(y_te, pred, target_names=LABELS)) # Precision·Recall
```

지표	값	이 값을 어떻게 읽는가
정확도	0.6548	기준선 0.5 보다 높다 — 균형 데이터라 이 비교가 성립한다
ROC-AUC	0.7258	0.5(찍기)보다 높다
Precision(양성)	0.79	대형이라 한 것 중 79%가 맞다
Recall(양성)	0.42	실제 대형의 42%만 잡아냈다

전부 다수 클래스로 찍었을 때의 정확도(기준선) 0.5

기준선은 0.5이며, 모델의 정확도 0.6548은 이 기준선 대비 약 15.5%p 높으므로 무작위 예측보다 유의미하게 개선된 성능이라고 해석할 수 있다. 그럼에도 ROC-AUC를 함께 보는 이유는, 정확도가 임계값 0.5 하나에서의 성능만 보여주는 반면 ROC-AUC는 모든 임계값에서의 분리 능력을 종합적으로 평가하기 때문입니다. 즉 임계값을 조정하더라도 모델 자체의 판별력이 얼마나 되는지 알 수 있는 지표입니다.

7 모델·입력정의서 저장

7-1 저장 코드

```
joblib.dump(model, '../02/model.pkl')

spec = {'title': TITLE, 'target_labels': LABELS,
        'accuracy': round(acc, 4), 'roc_auc': round(auc, 4),
        'features': FEATURES}
json.dump(spec, open('../02/feature_spec.json', 'w', encoding='utf-8'),
           ensure_ascii=False)
```

7-2 feature_spec.json 에 들어가는 4가지 키

각 입력마다 name · label · kind · options

이 파일이 왜 필요한가 03의 입력 화면이 이 파일을 읽어 폼을 자동으로 만든다

7-3 버전 일치 — 저장 환경과 서비스 환경

구축 요건

requirements.txt 의 버전은 모델을 학습한 환경과 **일치**해야 한다. 다르면 02 서버가

`ModuleNotFoundError` 로 뜨지 않는다.

라이브러리	학습 환경(01 출력)	requirements.txt	일치
python	3.11.0	3.11-slim	일치
numpy	2.4.6	2.4.6	일치
pandas	3.0.3	3.0.3	일치
scikit-learn	1.9.0	1.9.0	일치
joblib	1.5.3	1.5.3	일치

8 모델 API 서비스

8-1 엔드포인트 3개

메서드	경로	하는 일	응답에 무엇이 담기나
GET	/health	서버가 살아있는지	status UP, 모델 제목
GET	/spec	입력 정의 제공	title, target_labels, features
POST	/predict	예측(저장 안 함)	prediction, label, proba

8-2 예측 코드

```
df = pd.DataFrame([input_data])
return model.predict(df)[0], model.predict_proba(df)[0]
```

두 번째 함수가 돌려주는 것 클래스별 확률 배열 [0일 확률, 1일 확률]

요청 본문에 담은 값은 선택지의 인덱스 다 (문자열이 아니다). 그 이유 모델이 인코딩된 숫자로 학습했다. 문자열을 보내면 500 이 난다

8-3 오류 처리

없는 이력을 조회하면 404 Not Found 를 돌려줘야 한다. 그 코드 404

```
if row is None:
    raise HTTPException(status_code=404, detail='해당 예측 이력이 없습니다')
```

직접 호출해 확인한 결과 GET /predictions/999999 -> 404 해당 예측 이력이 없습니다

8-4 포트와 Swagger

이 모델 API 의 포트 8003 Swagger 주소 http://localhost:8003/docs

Swagger 에 보이는 엔드포인트 총 8 개

9 예측 이력 CRUD

9-1 CRUD 엔드포인트 5개

기능	메서드	경로	확인한 결과
C 생성	POST	/predictions	201, id=1
R 목록	GET	/predictions	200, 목록 2건
R 단건	GET	/predictions/{pid}	200, 단건
U 수정	PUT	/predictions/{pid}	200, 재예측됨
D 삭제	DELETE	/predictions/{pid}	200, deleted 1

9-2 수정 시 재예측 — 구축 요건

구축 요건

이력 수정 시 값만 바꾸는 것이 아니라 **새 입력으로 다시 예측**되어 결과가 갱신되어야 한다.

```
@app.put('/predictions/{pid}')
def update_prediction(pid: int, req: PredictRequest):
    result = run_model(req.features)    # 여기서 다시 예측한다
    ...
```

수정 전 라벨 소형 (확률 0.9604) → 수정 후 라벨 대형 (확률 1.0000)

이 함수를 호출하지 않으면 무엇이 잘못되나 입력만 바뀌고 결과는 옛 예측이 남아 이력이 서로 안 맞게 된다

9-3 저장 확인

이력이 저장되는 곳 02/store.py 의 sqlite DB (predictions 테이블)

삭제 후 다시 조회하면 404

10 컨테이너 통합·인증

10-1 docker compose 로 뜨는 컨테이너 5개

서비스	컨테이너명	포트	하는 일
DB	db-container	3330:3306	MySQL, 회원 정보
Redis	redis-container	6380:6379	캐시
ML	ml-container	8003:8003	FastAPI, model.pkl 예측
BN	bn-container	8080:8080	Spring Boot, 인증 + ML 중계
FN	fn-container	3000:80	React 화면

기동 명령 `docker compose up -d --build`

10-2 BN 이 ML 을 부르는 주소

```
@Value("${ml.url:http://ml-container : 8003}")
private String mlUrl;
```

localhost 가 아니라 컨테이너 이름을 쓰는 이유 컨테이너 안에서 localhost 는 자기 자신이다. 같은 네트워크의 서비스명으로 불러야 한다

10-3 인증 — 구축 요건

구축 요건

`/api/**` 경로는 **로그인해야 호출**되어야 한다(템플릿의 인증 설정 유지).

로그인 전에 `/api/spec` 을 부르면 상태코드 **401**

로그인 전에 예측 화면에 들어가면 어떻게 되나 예측 폼이 뜨지 않고 로그인 페이지로 이동한다

SecurityConfig 에서 이걸 강제하는 한 줄 `auth.anyRequest().authenticated()`

10-4 입력 화면 자동 생성 — 구축 요건

구축 요건

입력 화면은 `feature_spec.json` 을 읽어 자동 생성되어야 한다(하드코딩 불가).

```
{spec. features.map((f) => (  
  f.kind === 'num' ? <input type="number" .../>  
    : <select>{f. options.map((o, i) => <option value={i}>{o}  
</option>)}</select>  
  )})
```

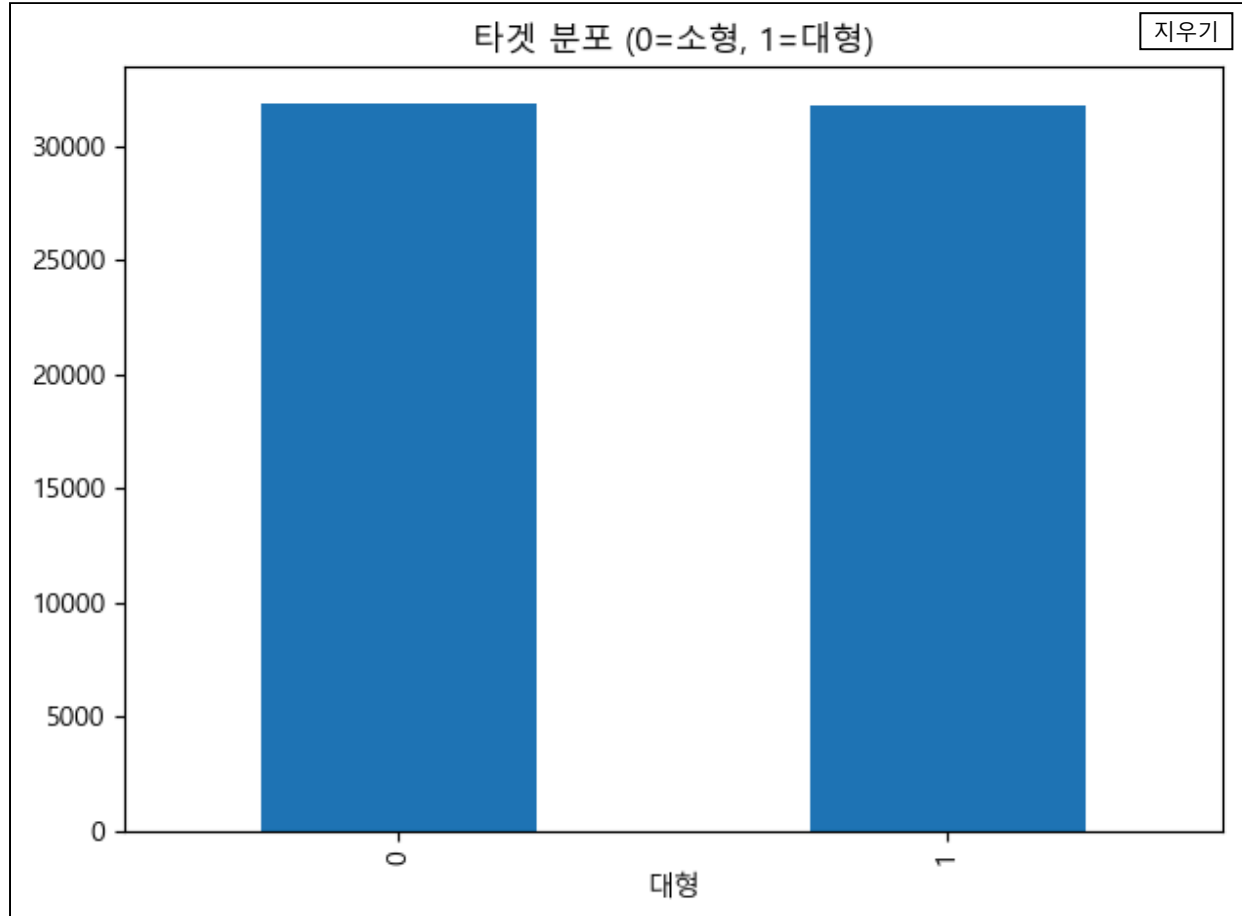
이 화면에 자동으로 생긴 입력 칸 3 개 (select 3 + number 0)

11-1 실행화면 캡처 (PPT 에 넣을 것)

넣는 법

캡처한 이미지 파일을 아래 상자에 **끌어다 놓는다**(상자를 눌러 파일을 골라도 된다). 넣은 이미지는 이 브라우저에 저장되고 인쇄·PDF 에도 함께 나온다.

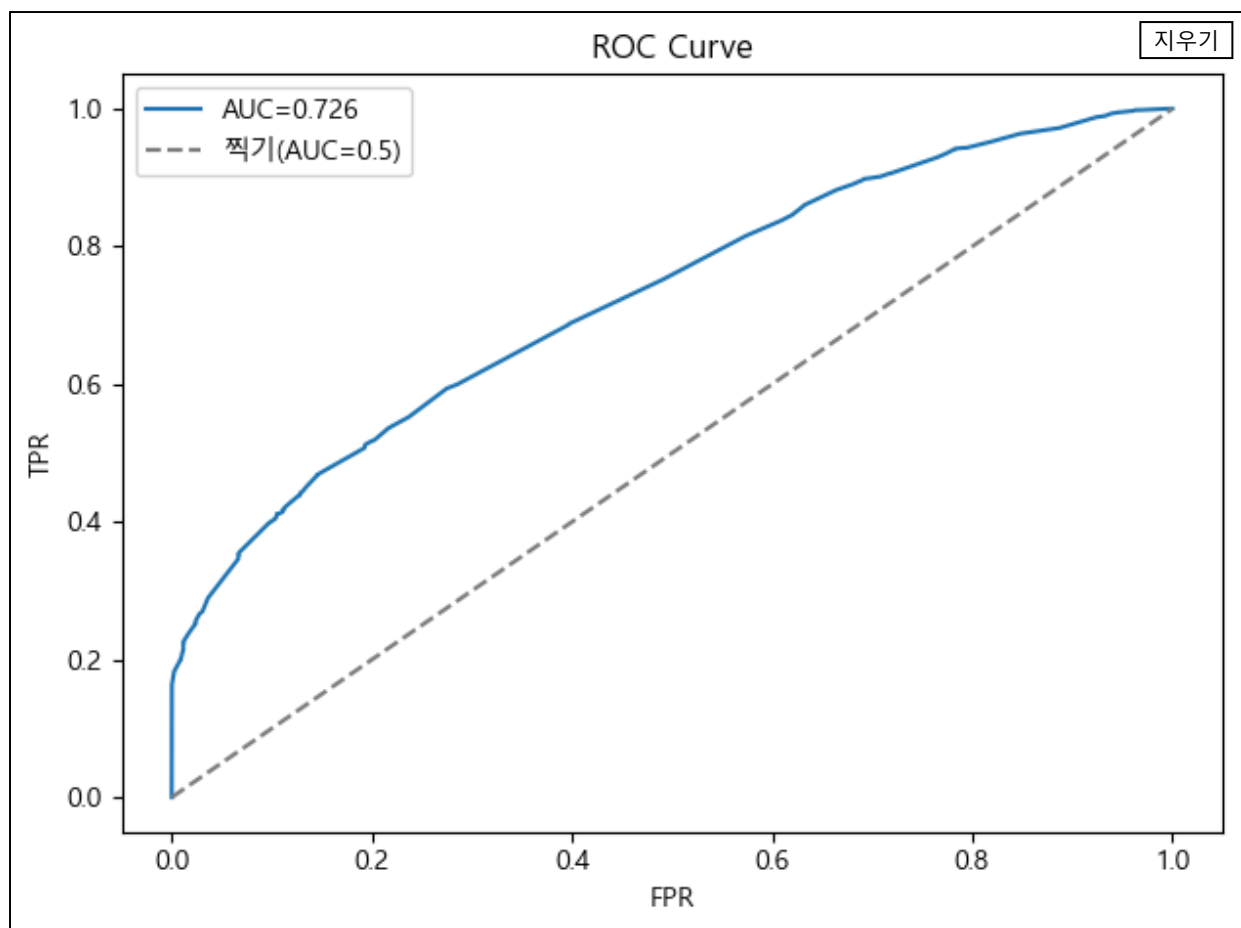
1. 품질 진단 타겟 분포



파일명 01-1_타겟분포.png

확인한 것 소형 31,886 vs 대형 31,761 — 49.90% 균형

2. ROC 곡선



파일명 01-2_ROC곡선.png

확인한 것 ROC-AUC 0.7258

3. Swagger

지우기

어선 규모(대형) 예측 ML API

0.1.0 OAS 3.1

[/openapi.json](#)

model.pkl 을 서빙하는 예측 엔드포인트 + 예측 이력 CRUD. Spring Boot(BN)가 호출한다.

default

GET /health Health

GET /spec Get Spec

POST /predict Do Predict

POST /predictions Create Prediction

GET /predictions List Predictions

GET /predictions/{pid} Get Prediction

PUT /predictions/{pid} Update Prediction

DELETE /predictions/{pid} Delete Prediction

Schemas

HTTPValidationError > Expand all object

PredictRequest > Expand all object

ValidationError > Expand all object

파일명 02-1_Swagger.png

확인한 것 엔드포인트 8개

4. 로그인

[MAIN](#) | [USER](#) | [PREDICT](#) | [JOIN](#) | [LOGIN](#) | [LOGOUT](#) |

지우기

LOGIN PAGE

USERNAME : student3

PASSWORD :

LOGIN

파일명 03-3_로그인.png
확인한 것 로그인해야 예측 가능

5. 예측 품(자동생성)

[MAIN](#) | [USER](#) | [PREDICT](#) | [JOIN](#) | [LOGIN](#) | [LOGOUT](#) |

지우기

어선 규모(대형) 예측

정확도 65.5% · ROC-AUC 0.7258

새 예측

선질(재질) :

FRP

용도 :

근해안강망어업

선적항(시도) :

강원

예측하고 저장

예측 이력 (0)

ID	결과	입력	시각	관리
이력이 없습니다. 위에서 예측을 해보세요.				

파일명 03-4_예측품_자동생성.png
확인한 것 spec 을 읽어 품이 자동 생성됨

6. 예측 결과

[MAIN](#) | [USER](#) | [PREDICT](#) | [JOIN](#) | [LOGIN](#) | [LOGOUT](#) |

지우기

어선 규모(대형) 예측

정확도 65.5% · ROC-AUC 0.7258

새 예측

선질(재질) : FRP

용도 : 근해안강망어업

선적항(시도) : 강원

예측하고 저장

예측 결과 : 대형

```
{
  "소형": 0,
  "대형": 1
}
```

예측 이력 (1)

ID	결과	입력	시각	관리
1	대형	{"material":1,"usage":0,"port":9}	2026-07-15T05:06:16	수정 삭제

파일명 03-5_고위험_예측결과.png

확인한 것 강·근해안강망어업·제주도 -> 대형 1.0000

7. 이력 수정 후

[MAIN](#) | [USER](#) | [PREDICT](#) | [JOIN](#) | [LOGIN](#) | [LOGOUT](#) |

지우기

어선 규모(대형) 예측

정확도 65.5% · ROC-AUC 0.7258

새 예측

선질(재질) : FRP

용도 : 근해안강망어업

선적항(시도) : 강원

예측하고 저장

예측 결과 : 대형

```
{
  "소형": 0,
  "대형": 1
}
```

예측 이력 (2)

ID	결과	입력	시각	관리
2	대형	{"material":1,"usage":0,"port":9}	2026-07-15T05:06:18	<button>수정</button> <button>삭제</button>
1	대형	{"material":1,"usage":0,"port":9}	2026-07-15T05:06:16	<button>수정</button> <button>삭제</button>

파일명 03-8_수정후.png

확인한 것 수정 시 새 입력으로 재예측될

8. 삭제 후

[MAIN](#) [USER](#) [PREDICT](#) [JOIN](#) [LOGIN](#) [LOGOUT](#)

지우기

어선 규모(대형) 예측

정확도 65.5% · ROC-AUC 0.7258

새 예측

선질(재질): FRP

용도: 근해안강망어업

선적항(시도): 강원

예측하고 저장

예측 결과 : 대형

```
{
  "소형": 0,
  "대형": 1
}
```

예측 이력 (1)

ID	결과	입력	시각	관리
1	대형	{"material":1,"usage":0,"port":9}	2026-07-15T05:06:16	수정 삭제

파일명 03-9_삭제후.png

확인한 것 삭제 후 이력이 줄어듦

11-2 예측 결과를 데이터 근거로 해석

이게 핵심이다

"모델이 그렇게 나왔다" 는 해석이 아니다. 품질 진단 단계에서 본 수치와 특성 중요도를 근거로 왜 그런 결과가 나왔는지 설명해야 한다.

높게 나온 입력 조합 선질=강 / 용도=근해안강망어업 / 선적항=제주도 → 결과 대형 (확률 1.0000)

낮게 나온 입력 조합 선질=FRP / 용도=해조류양식어업 / 선적항=충청북도 → 결과 소형 (확률 0.9604)

선질이 '강'인 선박은 대형 비율이 매우 높게 나와 모델이 대형일 확률로 강하게 말하고 있고, 선질이 'FRP'이고 용도도 '해조류양식어업'일 때의 선박은 두 가지 신호가 겹쳐 소형 비율이 매우 높게 나왔습니다.

두 케이스는 특정 케이스가 맞은 극단적인 경우이고, 임계값 0.5을 기준으로 전체적인 정확도를 보았을 때 0.65의 비율이 나온것 입니다.

11-3 특성 중요도와 대조

```
imp = pd.Series(model.feature_importances_,  
index=X.columns).sort_values(ascending=False)
```

1위 usage (0.632) 2위 port (0.222) 3위 material (0.146)

양성비율이 높았던 '재질'은 '강=100% 대형'이라는 완벽하게 깨끗한 카테고리를 갖고 있었는데도 중요도는 3위(0.146)로 가장 낮습니다. 반면 '용도'는 1위(0.632)입니다. 이러한 결과의 이유는 중요도는 '한 카테고리가 얼마나 깨끗한가'가 아니라 트리 전체에서 그 피처가 분기에 쓰이면서 불순도를 얼마나 많이, 얼마나 자주 줄였는가를 반영합니다.

11-4 모델의 한계

현재 X는 material-usage-port 3개, 그것도 전부 범주형 변수뿐입니다. 선박의 실제 크기를 결정하는 물리적 특성(길이,너비,깊이 등)이 전혀 들어있지 않고, '재질/용도/항구'는 크기와 간접적으로만 상관된 정보이기 때문에 설명력에 한계가 있습니다. 그 결과가 Recall 0.42로 나타난 것입니다. 재질·용도·항구만으로는 대형/소형이 애매하게 겹치는 선박이 많아서, 실제 대형의 절반 이상을 소형으로 잘못 분류합니다.